



QNX — ОС реального времени

QNX — Quick UNIX. «Небольшая» полуавтономная ОС:

- полнофункциональная ОС
- Платформа разработки программ QNX Software Development Platform (QNX SDP), на целевой машине (target) и на инструментальной

Концепция микроядра:

- потоки + процессы в изолированных АП + средства синхронизации
- Файловая система, сеть, GUI — изолированные процессы

Реальное время:

- Для встроенных систем (автомобилестроение, авиация)
- Простые алгоритмы управления процессором в РВ

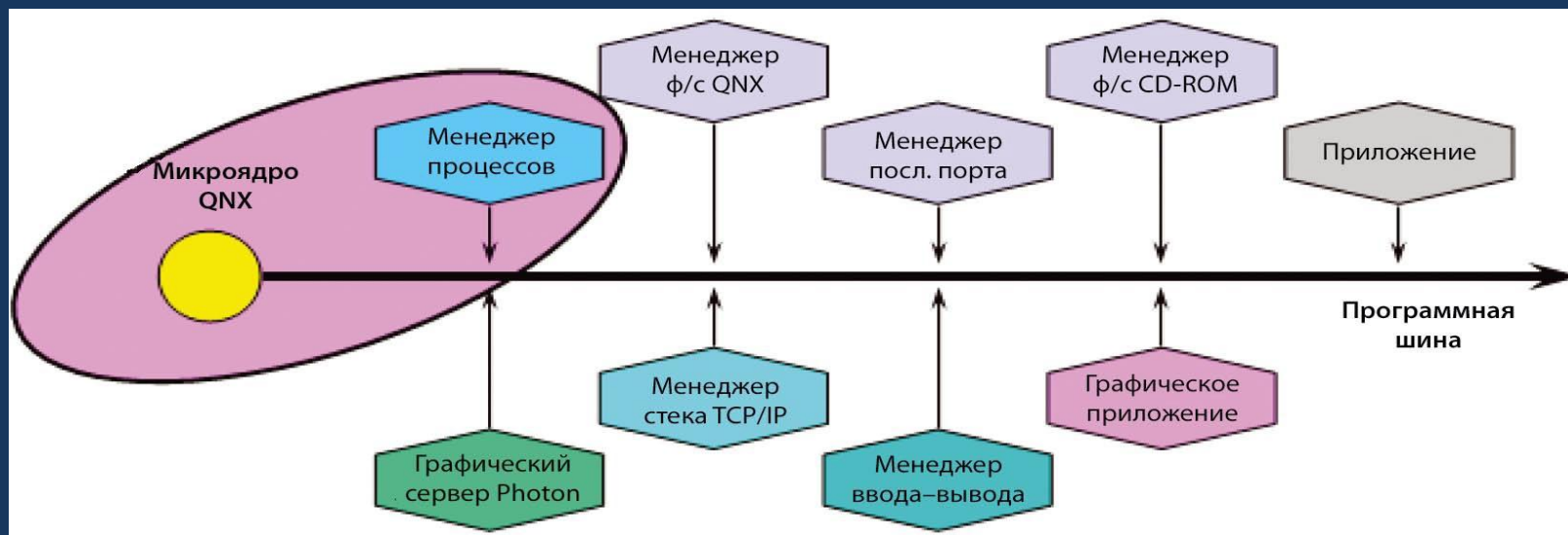
POSIX-совместимая. POSIX- программный интерфейс API прикладной программы в ОС, на уровне языка программирования и библиотек (Си).

Варианты установки:

- Раздел диска инструментального компьютера
- Целевой компьютер - контроллер



QNX — Микроядро



- обмен сообщениями
- управление потоками
- планирование (потоков)
- синхронизация (потоков)
- управление сигналами
- управление таймерами

Менеджер процессов — процесс, скомпонованный с микроядром



Потоки

Процесс — контейнер потоков, *не имеет самостоятельного значения в планировании процессора.*

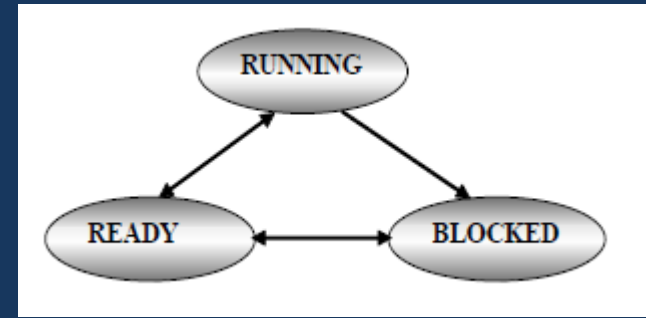
Атрибуты потоков

- `tid` — идентификатор потока — целое число, уникальное в пределах процесса
- `register context` — область для хранения контекста состояния задачи, используемая при блокировках и прерываниях потока
- `stack` — стек потока
- `signal mask` — собственная маска для сигналов — запрещающая или, наоборот, разрешающая прием и обработку сигналов
- `cancellation handlers` — функции, которые выполняются при завершении потока



Потоки — состояния блокировки

- блокировка по условной переменной (CONDVAR)
- начата, но не закончена процедура завершения потока (DEAD)
- поток прерван другим более приоритетным потоком (INTERRUPT);
- поток блокирован на входе в общую критическую секцию кода (MUTEX);
- поток «заснул» на некоторый интервал времени (NANOSLEEP);
- поток ждет ответа или подтверждения посылки при обмене сообщениями (REPLAY, NET_REPLAY, NET_SEND, RECEIVE);
- поток ожидает открытия семафора в заявленной группе потоков (SEM);
- поток ожидает появления сигнала (SIGSUSPEND, SIGWAITINFO, STOPPED);
- поток ожидает освобождения сопроцессора, выполняющего операции с плавающей точкой (WAITCTX);
- поток ожидает распределения затребованной памяти (WAITPAGE);
- поток ожидает завершения дочернего потока (WAITTREAD);





Потоки — синхронизация

Механизм синхронизации	Поддерживается между локальными процессами	Поддерживается между удаленными процессами
Мьютексы (Mutexes)	Yes	No
Условные переменные (Condvars)	Yes	No
Барьеры (Barriers)	No	No
Ждущие блокировки (Sleep on locks)	No	No
Блокировки Чтения/Записи (Reader/writer locks)	Yes	No
Семафоры (Semaphores)	Yes	Yes (только именованные)
FIFO-диспетчеризация (FIFO-scheduling)	Yes	No
Атомарные операции (Atomic operations)	Yes	No
Обмен сообщениями (Send/Receive/Reply)	Yes	Yes



Потоки — синхронизация

POSIX вызов	Вызов микроядра	Описание
pthread_mutex_init()	SyncTypeCreate()	Создать мьютекс
pthread_mutex_destroy()	SyncDestroy()	Уничтожить мьютекс
pthread_mutex_lock()	SyncMutexLock()	Захватить мьютекс
pthread_mutex_trylock()	SyncMutexLock()	Условное блокирование на мьютексе
pthread_mutex_unlock()	SyncMutexUnlock()	Освободить мьютекс
pthread_cond_init()	SyncTypeCreate()	Создать условную переменную
pthread_cond_destroy()	SyncDestroy()	Уничтожить условную переменную
pthread_cond_wait()	SyncCondvarWait()	Ожидать на условной переменной
pthread_cond_signal()	SyncCondvarSignal()	Обращение к условной переменной
pthread_cond_broadcast()	SyncCondvarSignal()	Широковещательное обращение к условной переменной



Синхронизация - мьютексы

Мьютекс — объект критической секции (неделимой операции), аналог `synchronized` в Java

```
int count = 0; /* Критическая область состоит из одной глобальной переменной
count */
/*****/
void* function1( void* arg ) // поток 1
{
    while( 1 ) { // бесконечный цикл
        pthread_mutex_lock( &mutex ); /* захват мьютекса (неудача приводит к блокировке те-
кущего потока) */
        count++; // изменение критической переменной (увеличение на 1)
        pthread_mutex_unlock( &mutex ); // Освобождение мьютекса
    }
    return 0;
}
/*****/
void* function2( void* arg ) // поток 2
{
    while( 1 ) { // бесконечный цикл
        pthread_mutex_lock( &mutex ); /* захват мьютекса (неудача приводит к блокировке те-
кущего потока) */
        count--; // изменение критической переменной (уменьшение на 1)
        pthread_mutex_unlock( &mutex ); // освобождение мьютекса
    }
    return 0;
}
```

Приоритет захватившего мьютекс повышается до приоритета захватывающего



Синхронизация — условные переменные

Ожидание до изменения переменной. При изменении — посылается сигнал.

```
int condition = 0; // условие, связанное с cond
int count = 0; // переменная критической секции
/*****/
int consume( void ) // поток-потребитель данных
{
    while( 1 )
    {
        pthread_mutex_lock( &mutex ); // захват мьютекса
        while( condition == 0 )
            pthread_cond_wait( &cond, &mutex ); /* блокировка потока по CONDVAR cond */
        printf( "Consumed %d\n", count );
        condition = 1;
        pthread_cond_signal( &cond ); // деблокировка потока, ожидающего cond
        pthread_mutex_unlock( &mutex ); // освобождение мьютекса
    }
    return( 0 );
}
/*****/
void* produce( void * arg ) // поток-производитель данных
{
    while( 1 )
    {
        pthread_mutex_lock( &mutex ); // захват мьютекса
        while( condition == 1 )
            pthread_cond_wait( &cond, &mutex ); /* блокировка потока по CONDVAR cond */
        printf( "Produced %d\n", count++ );
        condition = 0;
        pthread_cond_signal( &cond ); // деблокировка потока, ожидающего cond
        pthread_mutex_unlock( &mutex ); // освобождение мьютекса
    }
    return( 0 );
}
```




Синхронизация — семафоры

Семафор — синхронизатор доступа к ресурсу, имеющему N единиц

- $S(N)$
- $P(S)$: $S--$; if ($S < 0$) блокировка
- $V(S)$: $S++$; if ($S \geq 0$) деблокировка ожидающих
- при $N=1$ — мьютекс

```
int sem_init( sem_t * sem,      // указатель к объекту семафор
              int pshared,      // сфера действия семафора
              unsigned value );  // число пропускаемых потоков
```

```
int sem_post( sem_t * sem );
int sem_wait( sem_t * sem );
```

Функция	Описание
<code>sem_init()</code>	Инициализировать объект <i>семафор</i>
<code>int sem_post()</code>	Увеличить текущий счетчик
<code>int sem_wait()</code>	Уменьшить текущий счетчик
<code>sem_trywait()</code>	Уменьшить текущий счетчик, если его значение больше нуля
<code>sem_destroy()</code>	Уничтожить семафор



Синхронизация — блокировка RW

Читать можно параллельно. При блокировке по записи ждет завершения блокировок по чтению и монополюно блокирует ресурс

```
int pthread_rwlock_init(  
    pthread_rwlock_t * rwl,           // объект блокировки  
    const pthread_rwlockattr_t * attr ); // атрибуты
```

```
pthread_rwlock_rdlock( pthread_rwlock_t* rwl );
```

```
pthread_rwlock_wrlock(pthread_rwlock_t* rwl ),
```

Синхронизация — атомарные операции

Таблица 2.0

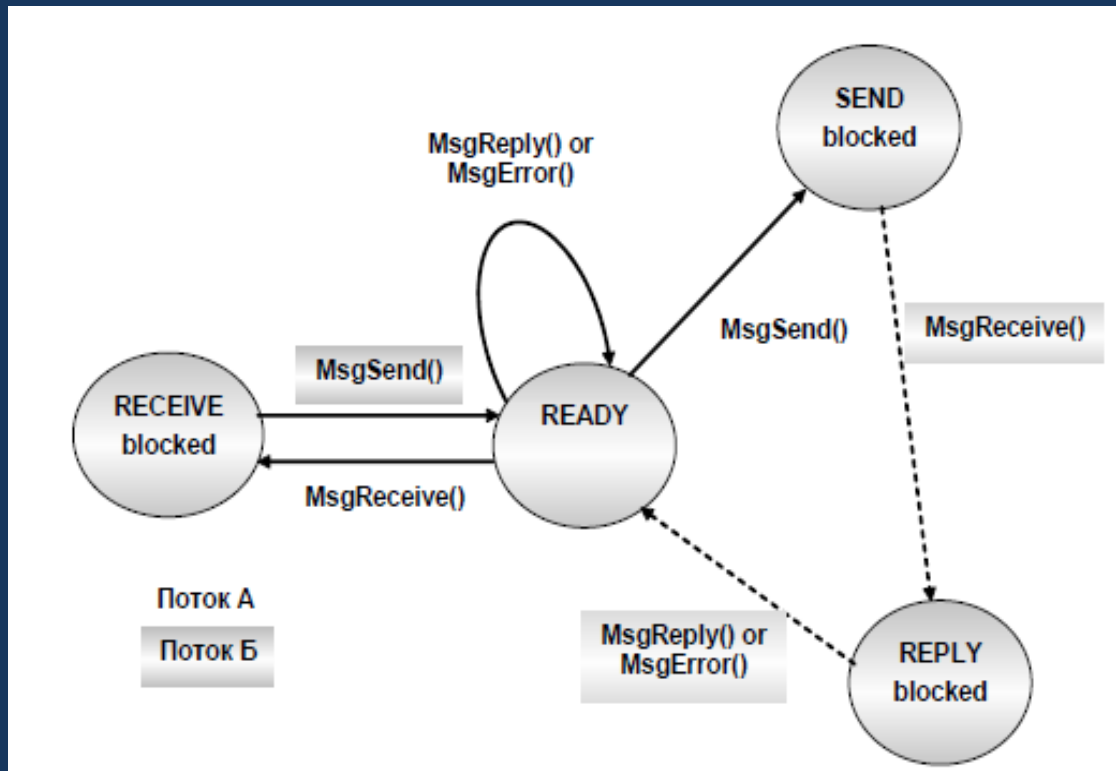
Функция		Описание
atomic_add(),	atomic_add_value()	Добавление значения
atomic_clr(),	atomic_clr_value()	Сброс бит
atomic_set(),	atomic_set_value()	Установка бит
atomic_sub(),	atomic_sub_value()	Вычитание значения
atomic_toggle(),	atomic_toggle_value()	Переключение бит

Все операции — адрес переменной



Синхронизация — сообщения

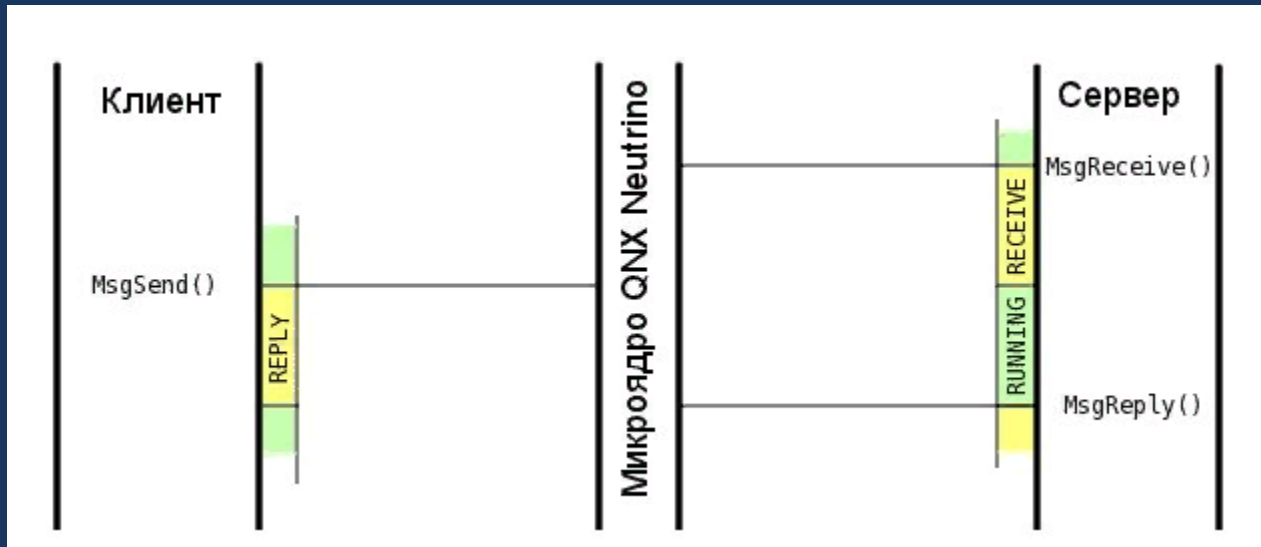
Прямая передача — память-память без буферизации



Функция	Описание
MsgSend()	Послать сообщение и блокироваться до получения ответа
MsgReceive()	Ожидать приема сообщения
MsgReply()	Ответить на сообщение
MsgError()	Возвратить код ошибки без передачи ответного сообщения



Синхронизация — сообщения



Блокировки SEND — передача, ожидание приемника, RECEIVE — приемник, ожидание передачи, REPLY — ожидание ответа на SEND



Тайм-ауты блокировок

```
#include <time.h>
extern int timer_timeout(
    clockid_t id,           // тип системных часов
    int flags,              // флаги, определяющие область действия тайм-аута
    const struct sigevent* notify, // уведомление о событии
    const struct timespec* ntime, // новое время тайм-аута
    struct timespec* otime );    // старое время тайм-аута
```

flags

Константа	Блокирующие состояния	Порождающие вызовы
_NTO_TIMEOUT_CONDVAR	STATE_CONDVAR.	SyncCondvarWait()
_NTO_TIMEOUT_JOIN	STATE_JOIN.	ThreadJoin()
_NTO_TIMEOUT_INTR	STATE_INTR.	InterruptWait()
_NTO_TIMEOUT_MUTEX	STATE_MUTEX.	SyncMutexLock()
_NTO_TIMEOUT_RECEIVE	STATE_RECEIVE.	MsgReceive()
_NTO_TIMEOUT_REPLY	STATE_REPLY.	MsgSend()
_NTO_TIMEOUT_SEM	STATE_SEM.	SyncSemWait()
_NTO_TIMEOUT_SEND	STATE_SEND.	MsgSend()
_NTO_TIMEOUT_SIGSUSPEND	STATE_SIGSUSPEND.	SignalSuspend()
_NTO_TIMEOUT_SIGWAITINFO	STATE_SIGWAITINFO.	SignalWaitinfo()



Тайм-ауты блокировок

Уведомление о тайм-ауте

Варианты уведомлений	Описание
SIGEV_INTR	Прерывание
SIGEV_PULSE	Импульс
SIGEV_SIGNAL	Сигнал
SIGEV_SIGNAL_CODE	Сигнал реального времени
SIGEV_SIGNAL_THREAD	Сигнал к потоку
SIGEV_UNBLOCK	Деблокировка потока

```
event.sigev_notify = SIGEV_UNBLOCK; // прямая установка типа уведомления
```

```
timeout.tv_sec = 10;
```

```
timeout.tv_nsec = 0;
```

```
timer_timeout( CLOCK_REALTIME,  
              _NTO_TIMEOUT_SEND | _NTO_TIMEOUT_REPLY,  
              &event, &timeout, NULL );
```

```
MsgSendv( coid, NULL, 0, NULL, 0 );
```



Тайм-ауты блокировок

Уведомление о тайм-ауте

Варианты уведомлений	Описание
SIGEV_INTR	Прерывание
SIGEV_PULSE	Импульс
SIGEV_SIGNAL	Сигнал
SIGEV_SIGNAL_CODE	Сигнал реального времени
SIGEV_SIGNAL_THREAD	Сигнал к потоку
SIGEV_UNBLOCK	Деблокировка потока

```
event.sigev_notify = SIGEV_UNBLOCK; // прямая установка типа уведомления
```

```
timeout.tv_sec = 10;
```

```
timeout.tv_nsec = 0;
```

```
timer_timeout( CLOCK_REALTIME,  
              _NTO_TIMEOUT_SEND | _NTO_TIMEOUT_REPLY,  
              &event, &timeout, NULL );
```

```
MsgSendv( coid, NULL, 0, NULL, 0 );
```



Коммуникации потоков

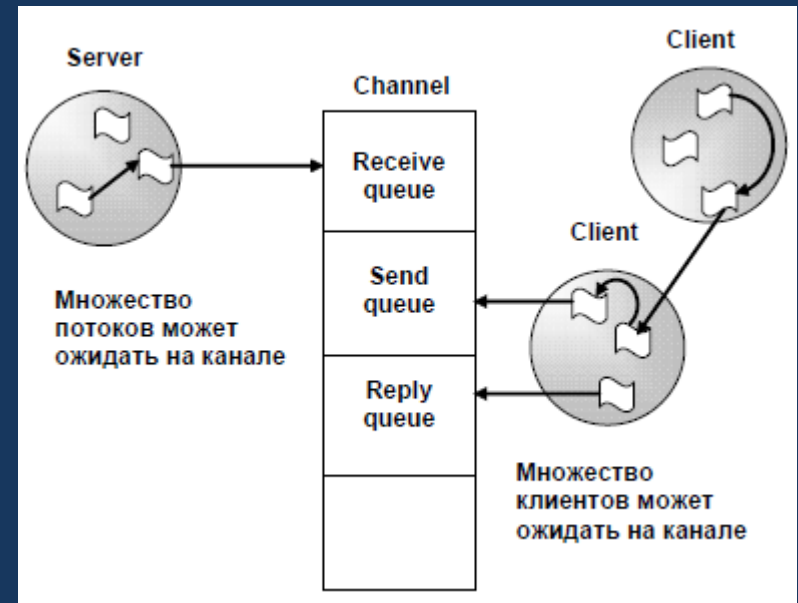
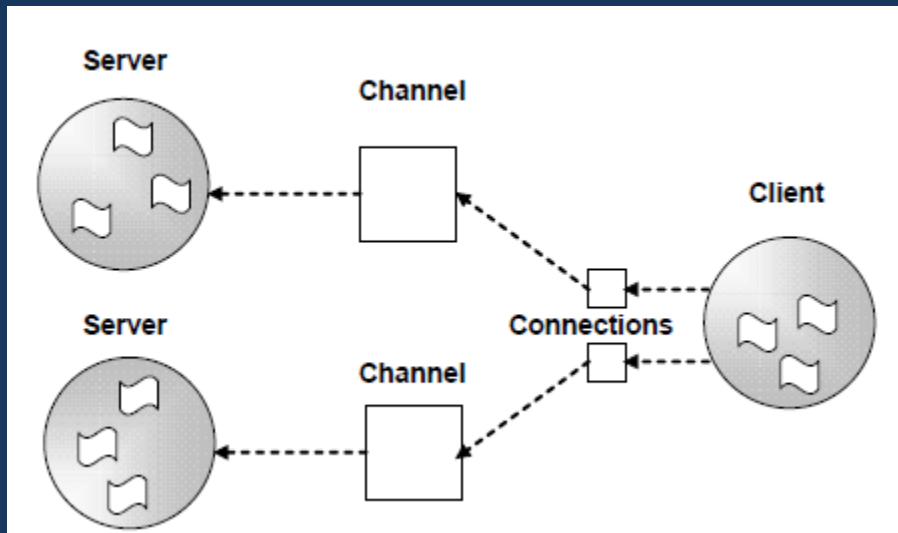
Формы взаимодействия	Область реализации
Обмен синхронными сообщениями	Ядро
Импульсы	Ядро
Сигналы	Ядро
Обмен асинхронными сообщениями	Ядро
POSIX очереди сообщений	Внешний процесс
Разделяемая память	Администратор процессов
Неименованные программные каналы (PIPEs)	Внешний процесс
Именованные программные каналы (FIFOs)	Внешний процесс



Синхронные сообщения

- Сервер создает канал
- Клиент создает соединение

Очереди потоков в канале



Функция	Описание
ChannelCreate()	Создать канал для приема сообщений
ChannelDestroy()	Уничтожить канал
ConnectAttach()	Создать соединение для передачи сообщения
ConnectDetach()	Уничтожить соединение



Синхронные сообщения

```
void *server() // поток сервера
{
int rcvd; // идентификатор принятого сообщения
char receive_buf[25]; // буфер для приема сообщения
char reply_buf[25]; // буфер для приема ответа
chid=ChannelCreate(0); // создание канала
sleep(1);
rcvd=MsgReceive(chid, &receive_buf, sizeof(receive_buf), NULL); // прием сообщения
printf ("Server thread: message <%s> has received \n", &receive_buf);
strcpy(reply_buf, "Strong answer from Server");
MsgReply(rcvd, 1500052, &reply_buf, sizeof(reply_buf)); // передача ответа
ChannelDestroy(chid); // уничтожить канал
pthread_exit(NULL);
}
/*****/
```

```
void *client() // поток клиента
{
int coid; // идентификатор соединения
int status; // статус ответного сообщения ( в данном случае 1500052)
pid_t PID;
char send_buf[25]; // буфер для передаваемого сообщения
char reply_buf[25]; // буфер для ответного сообщения
PID=getpid(); // получить программный идентификатор процесса
coid=ConnectAttach(chid,PID,1,0,0); // создать соединение
strcpy(send_buf, "It is very simple example");
```

```
/* передача сообщения */
status=MsgSend(coid,&send_buf, sizeof(send_buf), &reply_buf, sizeof(reply_buf));
printf ("Client thread: message <%s> has received \n", &reply_buf);
ConnectDetach(coid); // уничтожить соединение
pthread_exit(NULL);
}
```

```
/*****/
```

```
int main() { // главный поток
pthread_t server_tid, client_tid;
pthread_create(&server_tid, NULL, &server, NULL); // создать поток сервера
pthread_create(&client_tid, NULL, &client, NULL); // создать поток клиента
pthread_join(client_tid,NULL);
return 0;
}
```



Синхронные сообщения

```
void *server() // поток сервера
{
int rcvd; // идентификатор принятого сообщения
char receive_buf[25]; // буфер для приема сообщения
char reply_buf[25]; // буфер для приема ответа
chid=ChannelCreate(0); // создание канала
sleep(1);
rcvd=MsgReceive(chid, &receive_buf, sizeof(receive_buf), NULL); // прием сообщения
printf ("Server thread: message <%s> has received \n", &receive_buf);
strcpy(reply_buf, "Strong answer from Server");
MsgReply(rcvd, 1500052, &reply_buf, sizeof(reply_buf)); // передача ответа
ChannelDestroy(chid); // уничтожить канал
pthread_exit(NULL);
}
/*****/
```

```
void *client() // поток клиента
{
int coid; // идентификатор соединения
int status; // статус ответного сообщения ( в данном случае 1500052)
pid_t PID;
char send_buf[25]; // буфер для передаваемого сообщения
char reply_buf[25]; // буфер для ответного сообщения
PID=getpid(); // получить программный идентификатор процесса
coid=ConnectAttach(chid,PID,1,0,0); // создать соединение
strcpy(send_buf, "It is very simple example");
```

```
/* передача сообщения */
status=MsgSend(coid,&send_buf, sizeof(send_buf), &reply_buf, sizeof(reply_buf));
printf ("Client thread: message <%s> has received \n", &reply_buf);
ConnectDetach(coid); // уничтожить соединение
pthread_exit(NULL);
}
```

```
*****/
```

```
int main() { // главный поток
pthread_t server_tid, client_tid;
pthread_create(&server_tid, NULL, &server, NULL); // создать поток сервера
pthread_create(&client_tid, NULL, &client, NULL); // создать поток клиента
pthread_join(client_tid,NULL);
return 0;
}
```



Идентификация каналов

Регистрация имен процессов и создание каналов

Функция	Описание
<code>name_attach()</code>	Регистрировать имя процесса и создать канал
<code>name_open()</code>	Локализовать сервер и получить идентификатор соединения
<code>name_close()</code>	Уничтожить соединение с каналом сервера
<code>name_detach()</code>	Удалить имя процесса из пространства имен и созданный канал



Импульсы

Импульс — короткое сообщение, передаваемое без ожидания,
идентификатор клиента = 0 в `int rvc = Receive(...)`

```
struct _pulse {  
    uint16_t      type;  
    uint16_t      subtype;  
    int8_t        code;    // код импульса  
    uint8_t       zero[3];  
    union sigval   value;   // значение импульса  
    int32_t       scoid;    // идентификатор серверного соединения  
};
```

```
int MsgSendPulse ( int coid,    // идентификатор соединения  
                  int priority, // приоритет импульса  
                  int code,     // 8-битный код  
                  int value );  // 32-битное значение
```



Сигналы

Сигнал — асинхронное сообщение (формат импульса), способное деблокировать потоки и вызывать асинхронный код в процессе (аналог прерывания)

Вызовы микро-ядра	POSIX-вызовы	Описание
SignalKill()	kill(), pthread_kill(), raise(), sigqueue()	Послать сигнал к потоку, процессу или группе процессов
SignalAction()	sigaction(), signal()	Определить действие, которое нужно выполнить при приеме сигнала
SignalProcmask()	sigprocmask(), pthread_sigmask()	Изменить маску блокировки сигналов для потока
SignalSuspend()	sigsuspend(), pause()	Блокировать поток, пока сигнал обрабатывается
SignalWaitinfo()	sigwaitinfo()	Ожидать сигнал и вернуть информацию при приеме сигнала

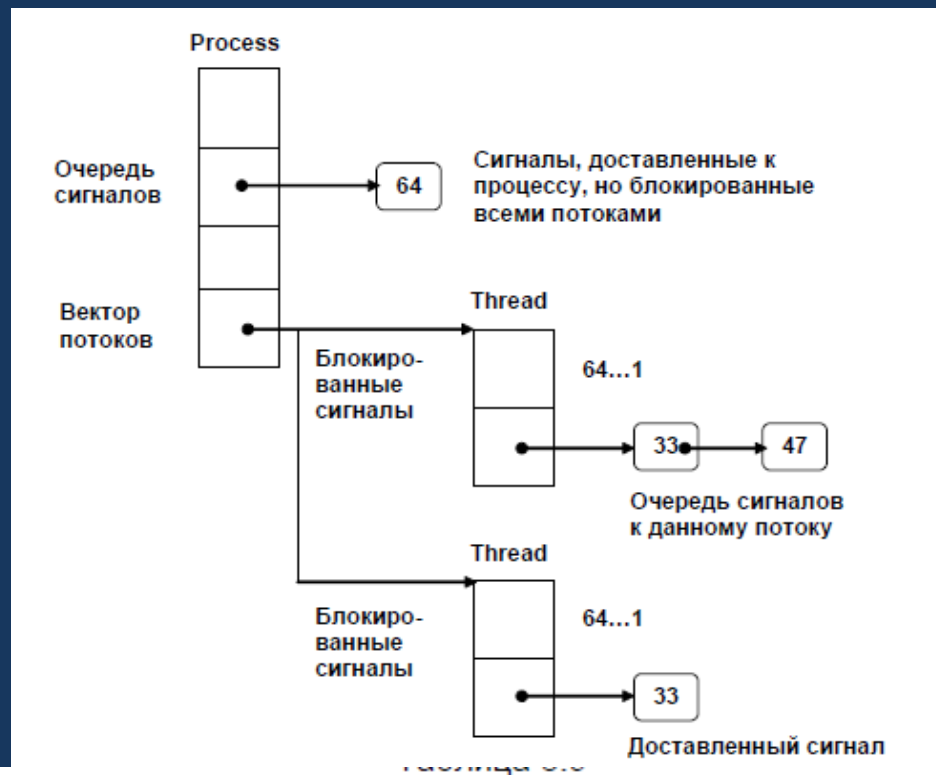
Виды реакции на сигнал:

- Игнорировать
- Маскировать (*блокировать*, сигнал сохраняется до снятия маски)
- Задать функцию обработки



Сигналы

Иерархическая система обработки сигнала для процесса и потоков



Функция	Назначение
sigemptyset()	Очистить список обрабатываемых сигналов
sigaddset()	Добавить сигнал в список
sigdelset()	Удалить сигнал из списка
sigfillset()	Добавить в список все сигналы
sigismember()	Проверить наличие сигнала в списке

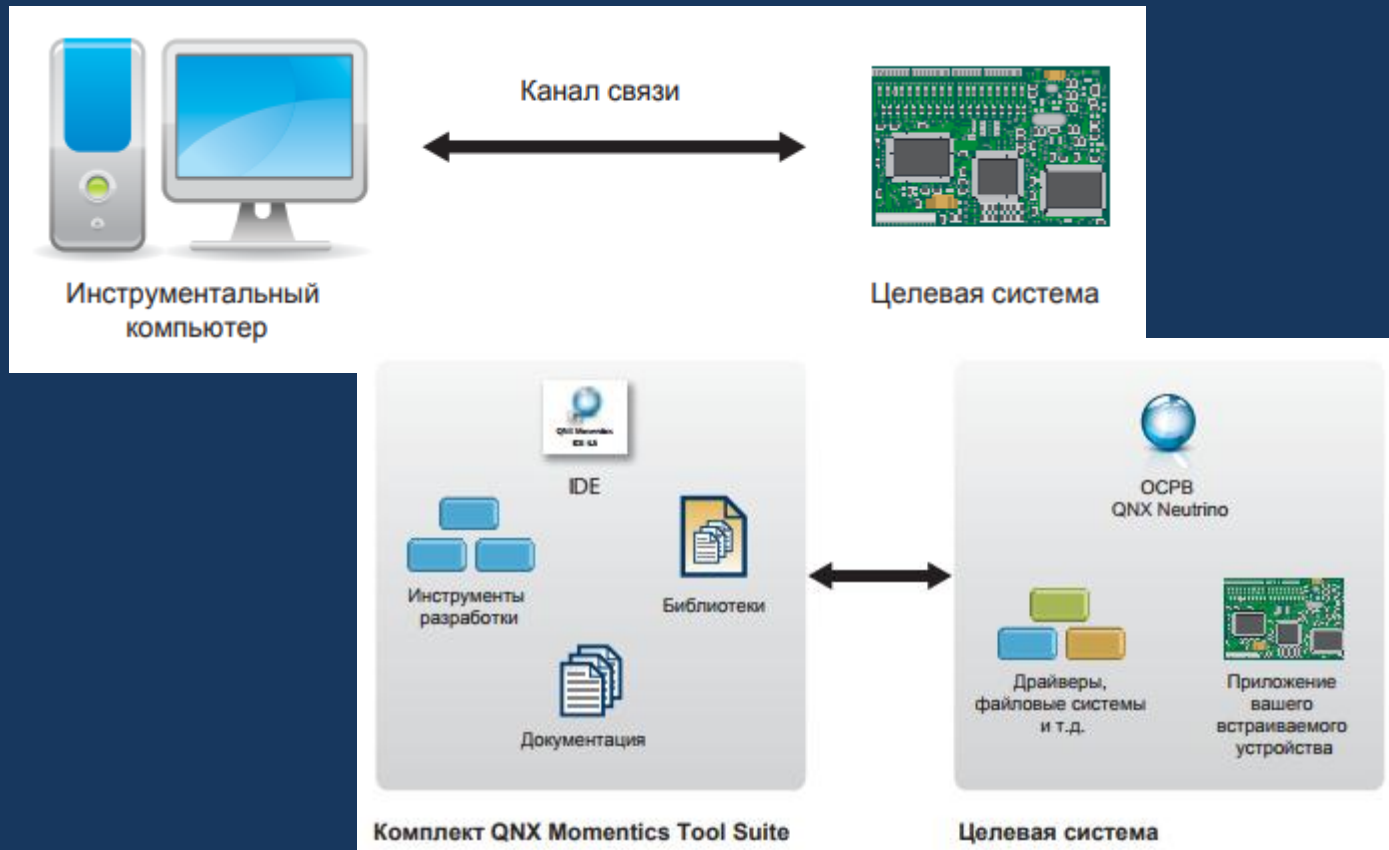


QNX — обработка прерываний

- Импульс (SIGEV_PULSE). Для приёма импульса применяются системные вызовы `MsgReceive()`, `MsgReceivePulse()`. Все уведомления о прерываниях ставятся в очередь и обрабатываются в соответствии с приоритетами импульсов. Несколько потоков могут ожидать импульса, что делает эту схему весьма гибкой
- Сигнал (SIGEV_SIGNAL, SIGEV_SIGNAL_CODE). Применяется в паре с одной из многочисленных библиотечных функций (`pause()`, `Sigwait()`, `Signal-Suspend()`, `SignalWaitinfo()`,...), предназначенных для приёма и обработки сигналов. Имеется возможность выбрать, какие уведомления ставить в очередь на обработку. В сочетании с `sigwaitinfo()` работает несколько быстрее, чем импульс
- Событие-прерывание (SIGEV_INTR). Самый простой и быстрый способ. Применяется вместе с системным вызовом `InterruptWait()`. Только один поток может ожидать это событие - тот, который вызвал `InterruptAttach()` или `InterruptAttachEvent()`. Ядро не ставит в очередь более одного уведомления



QNX SDP — Momentics Tool Suite





QNX SDP — Momentics Tool Suite

Debug - AQNXProject/AQNXProject.c - QNX Momentics IDE

File Edit Refactor Navigate Search Project Run Window Help

Debug <terminated> AQNXProject [C/C++ QNX QConn (IP)]
<terminated, exit value: 0> /tmp/AQNXProject on TestTarget pid 622630 (3/26/09 9:11 AM)

AQNXProject.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char ** argv){
    char * ptr = NULL;
    ptr = malloc(12);
    strcpy(ptr, "Hello World!");
    return 0;
}
```

Outline Disassembly

Console Tasks Problems Executables Mudflap Violations

C:\jde-4.6-workspace\AQNXProject\output.mudflap

Violation	Location	Thread	Time
write out of bounds (strcpy dest)	(strcpy dest)	1	09:30.57.185
backtrace	0xb8207c02		09:30.57.185
malloc region[12] (heap) accessed at [0..12]	malloc region		
alloc backtrace	0xb820758e	1	09:30.57.185
[0] libmudflapth.so.0 (__mf_register)	libmudflapth.so.0(__mf_register+0x3e) [0xb820758e]		
[1] libmudflapth.so.0 (__real_malloc)	libmudflapth.so.0(__real_malloc+0xba) [0xb8208b6a]		
[2] AQNXProject (main)	AQNXProject(main+0x18) [0x8049004]		
[3] libmudflapth.so.0 (__wrap_main)	libmudflapth.so.0(__wrap_main+0x41) [0xb8207659]		
memory leak of size 20			
alloc backtrace	0xb820758e	1	09:30.57.182
[0] libmudflapth.so.0 (__mf_register)	libmudflapth.so.0(__mf_register+0x3e) [0xb820758e]		
[1] libmudflapth.so.0 (__real_malloc)	libmudflapth.so.0(__real_malloc+0xba) [0xb8208b6a]		
[2] libc.so.3 (atexit)	libc.so.3(atexit+0x19) [0xb032ac99]		
[3] libc.so.3 (_init_libc)	libc.so.3(_init_libc+0x33) [0xb03641b3]		
malloc region[20] (heap) accessed at [0..20]	malloc region		
memory leak of size 20			



QNX SDP — Eclipse

debuggermemoryanalysis.bmp

File Edit Image Help

Debug - QNX® Momentics® Integrated Development Environment

File Edit Perspective Project Debug Window Help

Debug

- MallocErr (Attach To Remote Process)
- MallocErr at 10.1.0.195:8000(Running)
- Thread 1 (Suspended)
- a_bad_func() at MallocErr.c:22
- ! main() at MallocErr.c:25

Memory Events

- node440
- MallocErr(10305508)
- Pointer within malloc region, but outside of malloc data bounds
- a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:20]

Allocation Trace

Call	Pointer	Len	Where
malloc	0x0804B1C8	8	libc.so.2 (atexit+0x0000001D)0x8031BALS
malloc	0x0804D2C0	16..	(main+0x00000236)monstartup
malloc	0x0804B200	10	a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:15]
free	0x0804B200		a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:21]

Navigator Processes Debug

Breakpoints Inspector Variables Memory Watch Registers Display Memory Events

MallocErr.c

```
/*
 * MallocErr.c
 *
 * A single app that will generate a memory error.
 */

void a_bad_func() {
    char *ptr;

    ptr = malloc(16);

    /* We should get a memory error on the next line because we are strcpy'ing
     * past the end of the buffer
     */
    strcpy(ptr, "1234567890");
    free(ptr);
}

int main(int argc, char **argv) {
    a_bad_func();
    return 0;
}
```

outline

- malloc.h
- stdio.h
- stdlib.h
- string.h
- a_bad_func()
- main()

Tasks (Filter showing 1 of 02 items)

C	Description	Resource	In Folder	Location
☒	Pointer within malloc region, but outside of malloc data bounds	MallocErr.c	eclipse-test	line 20 in a_bad..

Tasks Console GDB Output



QNX SDP — Eclipse

debuggermemoryanalysis.bmp

File Edit Image Help

Debug - QNX® Momentics® Integrated Development Environment

File Edit Perspective Project Debug Window Help

Debug

- MallocErr (Attach To Remote Process)
- MallocErr at 10.1.0.195:8000(Running)
- Thread 1 (Suspended)
- a_bad_func() at MallocErr.c:22
- ! main() at MallocErr.c:25

Memory Events

- node440
- MallocErr(10305508)
- Pointer within malloc region, but outside of malloc data bounds
- a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:20]

Allocation Trace

Call	Pointer	Len	Where
malloc	0x0804B1C8	8	libc.so.2 (atexit+0x0000001D)0x8031BALS
malloc	0x0804D2C0	16..	(main+0x00000236)monstartup
malloc	0x0804B200	10	a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:15]
free	0x0804B200		a_bad_func [thoma/pgraves/workspace/eclipse-test/MallocErr.c:21]

Navigator Processes Debug

Breakpoints Inspector Variables Memory Watch Registers Display Memory Events

MallocErr.c

```
/*
 * MallocErr.c
 *
 * A single app that will generate a memory error.
 */

void a_bad_func() {
    char *ptr;

    ptr = malloc(16);

    /* We should get a memory error on the next line because we are strcpy'ing
     * past the end of the buffer
     */
    strcpy(ptr, "1234567890");
    free(ptr);
}

int main(int argc, char **argv) {
    a_bad_func();
    return 0;
}
```

outline

- malloc.h
- stdio.h
- stdlib.h
- string.h
- a_bad_func()
- main()

Tasks (Filter showing 1 of 02 items)

C	Description	Resource	In Folder	Location
☒	Pointer within malloc region, but outside of malloc data bounds	MallocErr.c	eclipse-test	line 20 in a_bad..

Tasks Console GDB Output



QNX Photon — администратор GUI

